CEPARCO Project Proposal

Group 1 Members:

1. Billones, Francis Hubert D.
2. Dollentas, Raine

1. Abstract

Zhang's six-frame alignment algorithm compares short DNA reads against reference proteins while accounting for all six reading frames and for frameshifts caused by sequencing errors. Its dynamic-programming approach is computationally expensive, since every read must be aligned against tens of thousands of proteins in a reference proteome. A prior CUDA implementation by Arceta et al. parallelized the algorithm within each DP matrix using a tiled anti-diagonal wavefront and achieved mean speedups of 3.5x over sequential execution. However, because DNA reads are short, each matrix is small, leaving much of the GPU underutilized. This project proposes an alternative mapping based on inter-task parallelism: each read-protein alignment is treated as an independent task assigned to its own GPU worker, and all tasks are processed concurrently. The implementation will be validated against the outputs of the prior CUDA implementation on the same fruit fly and mouse-ear cress datasets, and its performance will be compared against both a sequential baseline and the prior implementation on identical hardware.

2. Description of proposed project

Sequence alignment is a fundamental problem in bioinformatics. Modern sequencer machines process DNA into millions of short fragments called "reads", typically around 150 nucleotides long. A common research question is identifying which known protein a read came from, and where. It requires comparing each read against a reference proteome (the complete set of known proteins for an organism), which can contain thousands of sequences.

Zhang's six-frame alignment algorithm [4], specifically its local-alignment variant [2][3], uses a dynamic-programming approach. The algorithm can be structured in a way that benefits highly from parallelization. Prior work by Arceta et al. [1] implemented Zhang's six-frame algorithm in CUDA and reported mean speedups of about 3.5x over a serial implementation.

We plan to build on this work and show that a different approach of mapping the problem onto the GPU can achieve higher throughput.

a. Inputs to the project

We will use the same datasets as Arceta et al. [1]:

- Reference proteomes (FASTA) from UniProt: *Drosophila melanogaster* (fruit fly) and *Arabidopsis thaliana* (mouse-ear cress) [6]
- DNA reads (FASTA) from the European Nucleotide Archive [1]
- BLOSUM62 substitution matrix [5]
- Gap-open, gap-extend, and frameshift penalties identical to their implementation

b. Proposed Process

i. Processes to be parallelized

The scoring phase, which fills the DP matrices, dominates the runtime and will be parallelized with CUDA. Each (strand, protein) combination is an independent alignment task with its own DP matrix, giving roughly 43k tasks per read for the fruit-fly proteome and 78k for mouse-ear cress. We will parallelize primarily across these independent tasks, and also within each task using a warp-level wavefront.

ii. Existing Implementations

Lim et al. [2], [3] implemented the modified three-frame and six-frame algorithms using AVX2. They used a staircase-like traversal, and achieved about a 3x speedup over sequential execution.

Arceta et al. [1] processed one read-protein pair at a time. Each DP matrix is divided into 32x32 tiles, which are processed along tile anti-diagonals with one kernel launch per tile anti-diagonal. Within a tile, a 1,024-thread block sweeps the cell anti-diagonals using block-level synchronization. They report mean speedups of 3.51x and 3.44x on their two datasets, with a maximum of about 6.4x.

From our analysis of the published source code [7], we identify the following factors limiting utilization:

1. Each read is only 150 nt long, so each matrix is about 5 tiles tall. At most about 5 thread blocks are active, which is far fewer than a modern GPU can run concurrently.
2. Each 32-thread warp maps to a single tile row. At most only one thread per warp is active at each step.

We replace the tiled intra-matrix approach with an inter-task design. Each warp will be responsible for an entire alignment task, and all tasks for a read are processed in a single kernel launch. Since tens of thousands of tasks exist, this design keeps the GPU fully occupied.

iii. Plans on how to parallelize the proposed process

As mentioned above, our approach centers on inter-task parallelism. Each read-protein-strand combination is an independent alignment task, and a single read already requires tens of thousands of such tasks. Rather than parallelizing within one DP matrix, as in [1], we plan to assign each task to its own GPU worker (a thread or a warp) and process all tasks in a single kernel launch.

Because each thread/warp is now in charge of an alignment task, we will also order tasks by increasing length, such that bottlenecks from threads/warps working on long tasks can be avoided until necessary.

iv. Other processes in the project that will not be parallelized

- Reading and parsing FASTA input files and the BLOSUM62 matrix
- Computing the reverse complement of each read
- Building the query profile
- Selecting the top-scoring proteins from the per-task results

- Formatting and writing output

c. Outputs of the project

For each input read, the program outputs the top-5 matching proteins, ranked by alignment score. For each match it reports the protein identifier, the alignment score, the start and end positions of the aligned region, the strand, and the alignment itself. We compare these results with the outputs of the reference implementation [1] on identical inputs.

We will also report:

- Execution time of the sequential baseline, the reference CUDA implementation [1], and our implementation
- Speedup relative to the sequential baseline and to [1]
- Throughput in giga cell updates per second (GCUPs), the standard metric for DP alignment

3. Proposed target implementation platform of the proposed project

Experiments will be performed on Google Colab, using their T4 GPU runtime. Implementation will be in C/C++ and CUDA, compiled with nvcc on Linux.

Because Arceta et al. [1] reported results on an RTX 4070 Super, their timings are not directly comparable to measurements on Colab's T4. To ensure a fair comparison, we will port their codebase from Windows to Linux and re-run it on the same Colab hardware as our implementation.

4. Mid-project milestones

The timeline below serves as our mid-project milestones for the proposed project:

Milestone #1: Week 7-8: Baseline porting of [1] to Linux on Colab, and sequential C baseline implemented and validated

Milestone #2: Week 8-9: Thread-per-task CUDA kernel implemented and validated

Milestone #3: Week 10: Mid-project update demo

Milestone #4: Week 10-11: Unit testing, bug hunting and finalizing

Milestone #5: Week 11-12: Testing of correctness, performance, and speedup metrics

Milestone #6: Week 13: Complete project submission and demo

5. Mid-project updates

As of November 13, 2026, we have accomplished:

Milestone #1: Baseline porting of [1] to Linux on Colab, and sequential C baseline implemented and validated

Milestone #4: Unit testing, bug hunting and finalizing

Milestone #5: Testing of correctness, performance, and speedup metrics

We are moving **Milestone #2** to after the project update, the reason being that the proponents have determined that doing Milestones #4 and #5 first is better will make Milestone #2 easier to do. A much more passionate explanation is available in the video.

A summary of our changes:

- Refactored the codebase to be cross-platform compatible
- Restructured the codebase heavily, to isolate responsibilities
- Fixed bugs, 3 of which have to do with program correctness

Those 3 bugs were:

- The program hardcoded the size of the protein database, meaning that if the provided file was bigger than that size it would ignore the latter part of the file
- The program only printed alignments for DNA read 0, even though it was computing alignments for all provided DNA reads
- The program doubly-counted strands in six-frame mode

We pointed out just these bugs because they meant that our code output is now not 1:1 with the original codebase, even though our original goal was to just speedup the program and not affect its behavior. We are more than happy to communicate with the original authors in case these were in fact wrong.

6. References

[1] A. Z. Arceta, A. G. Mendoza, J. T. Tan, and R. L. Uy, “Accelerating Zhang’s six-frame alignment algorithm via hybrid SIMT wavefront parallelization on CUDA,” in *Proc. Workshop on Computation: Theory and Practice (WCTP 2025)*, Atlantis Press, 2026, pp. 403–417, doi: 10.2991/978-94-6239-638-8_20.

[2] S. V. Lim, S. E. Lim, C. L. Ting, A. E. Wong, and R. L. Uy, “SIMD implementation of local alignment-based six-frame alignment algorithm,” *Philippine Computing Journal*, vol. 17, no. 1, 2023.

[3] S. V. Lim, S. E. Lim, C. L. Ting, A. E. Wong, and R. L. Uy, “SIMD implementation of modified Zhang’s three-frame alignment algorithm,” in *Proc. IEEE 13th Int. Conf. HNICEM*, 2021, pp. 1–6, doi: 10.1109/HNICEM54116.2021.9731997.

[4] Z. Zhang, W. R. Pearson, and W. Miller, “Aligning a DNA sequence with a protein sequence,” *Journal of Computational Biology*, vol. 4, no. 3, pp. 339–349, 1997.

[5] S. Henikoff and J. G. Henikoff, “Amino acid substitution matrices from protein blocks,” *Proc. Natl. Acad. Sci. USA*, vol. 89, no. 22, pp. 10915–10919, 1992.

[6] The UniProt Consortium, “UniProt Proteomes: UP000000803 (*Drosophila melanogaster*) and UP000006548 (*Arabidopsis thaliana*),” https://www.uniprot.org/proteomes.

[7] A. Z. Arceta, A. G. Mendoza, and J. T. Tan, “Zhang’s Six-Frame in CUDA,” GitHub repository, https://github.com/Shiray427/Zhangs-Six-Frame-in-CUDA.